NAME:- ARYAN SINGH SIKARWAR

REG N0:- 24BCE0403

EXPERIMENT NO:- 2

DATE:- 4-FEB-2026

MPMC LAB

Aim (Statement)

To write and execute an **8051 assembly language subroutine** that copies data already transferred from **ROM to RAM**, by copying the contents of **RAM locations starting from 40H to RAM locations starting at 60H**, and to **single-step through the program and examine the RAM contents**.

Apparatus Required

1. **8051 Microcontroller** (e.g., AT89C51 / AT89S52)
2. **Microcontroller Trainer Kit** *or* **8051 Simulator Software** (Keil µVision / Proteus / EdSim51)
3. **Personal Computer (PC)**
4. **Assembler / IDE** (Keil µVision or equivalent)

5. **Power Supply** (if using hardware trainer kit)
6. **Connecting Cables** (for hardware setup)

Question 1

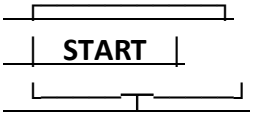
Aim

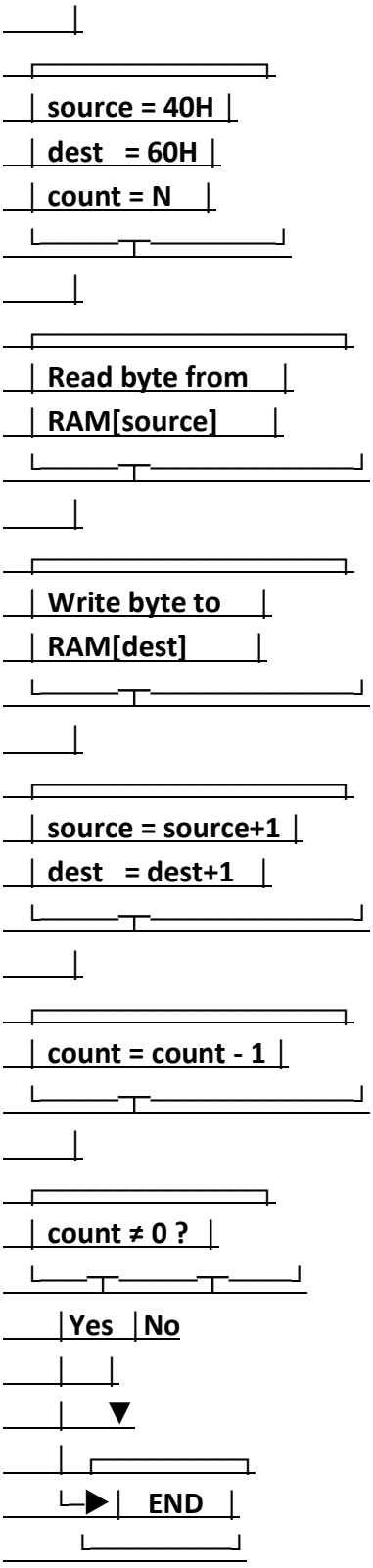
Question- 1
Add the following subroutine to the program 1, single-step through the subroutine and examine the RAM locations. After data has been transferred from ROM space into RAM, the subroutine should copy the data from RAM locations starting at 40H to RAM locations starting at 60H.

Algorithm (Step-by-Step)

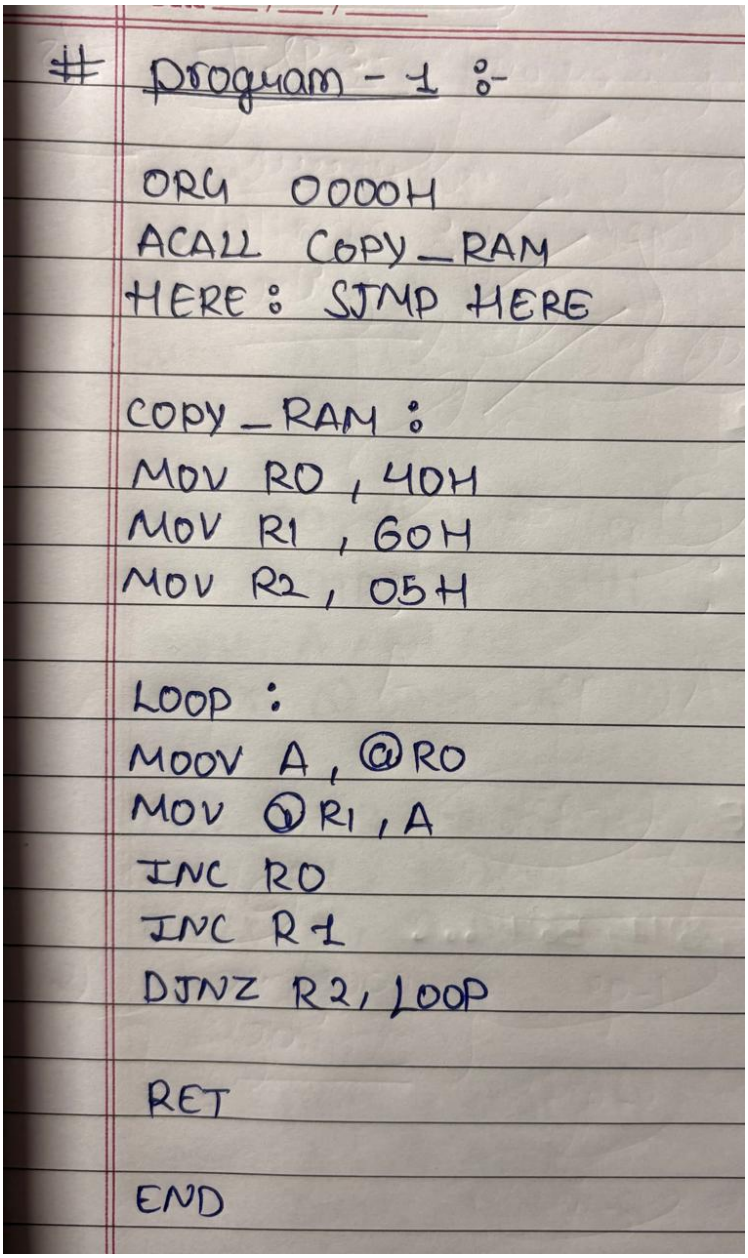
1. **Start**
2. Initialize:
    a. Source pointer = 40H
    b. Destination pointer = 60H
    c. Counter = number of bytes to be copied (N)
3. Read one byte from source RAM location
4. Store the byte into destination RAM location
5. Increment source pointer
6. Increment destination pointer
7. Decrement counter
8. If counter ≠ 0, repeat from Step 3
9. **End subroutine**
10. Return to main program

FLOWCHART:-




source = 40H
dest = 60H
count = N
Read byte from
RAM[source]
Write byte to
RAM[dest]
source = source+1
dest = dest+1
count = count - 1
count ≠ 0 ?
Yes
No
END

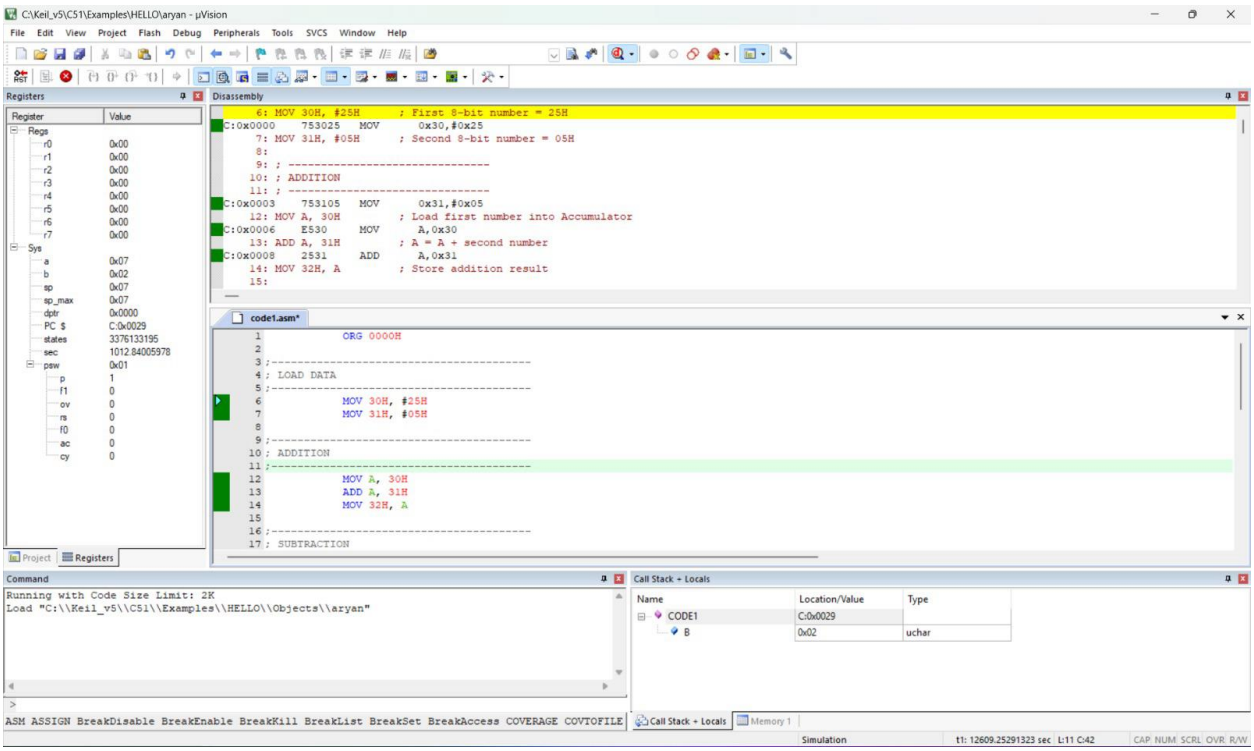
CODE:-(HAND WRITTEN)



Program - 1 :-

```
ORG 0000H
ACALL COPY_RAM
HERE: SJMP HERE

COPY_RAM:
MOV R0, 40H
MOV R1, 60H
MOV R2, 05H

LOOP:
MOOV A, @R0
MOV @R1, A
INC R0
INC R1
DJNZ R2, LOOP

RET

END
```

SCREENSHOT OF THE CODE:-



Observation / Inference / Result

Observation:

- Data initially stored at RAM locations starting from **40H** is successfully copied to RAM locations starting from **60H**.
- During single-step execution, registers **R0, R1, and R2** update correctly.
- Memory window confirms identical data at source and destination locations.

Inference:

- The subroutine executes correctly using indirect addressing and loop control.
- Data transfer within RAM is achieved efficiently using registers.

Result:

Thus, the **8051 ALP subroutine to copy data from RAM location 40H to 60H was successfully executed and verified using Keil software**.

Question 2

Aim (Statement)

To write and execute an **8051 Assembly Language Program (ALP)** to load values into registers **R0 to R4**, push these registers onto the **stack**, single-step through the program, and **examine the stack memory and Stack Pointer (SP) register** after execution of each instruction using **Keil software**.

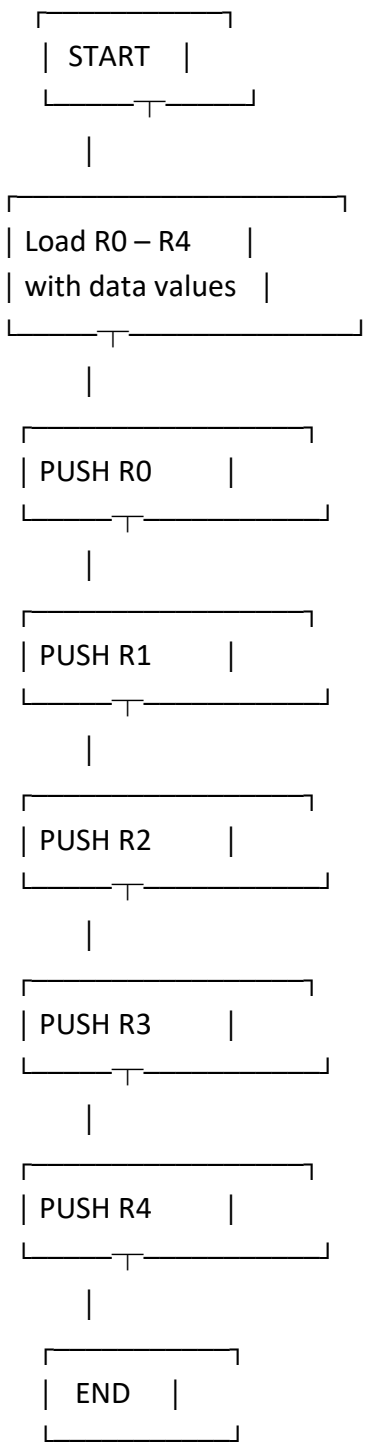
2. Apparatus Required

1. Personal Computer (PC)
2. Keil µVision Software
3. 8051 Microcontroller Simulator (Keil built-in simulator)
4. 8051 Instruction Set
5. Power supply (if hardware kit is used)

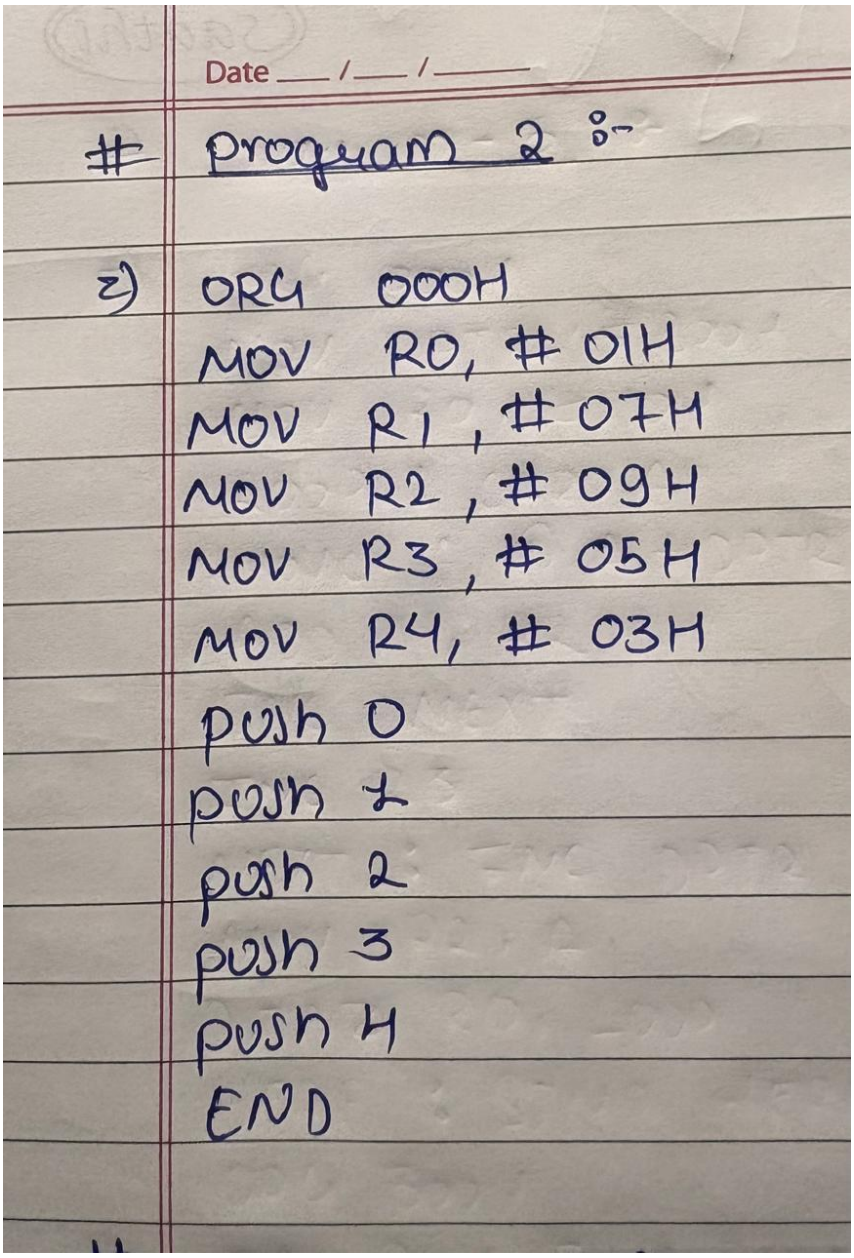
3. Algorithm

1. Start the program.
2. Initialize registers **R0 to R4** with known data values.
3. Push the contents of **R0** onto the stack.
4. Push the contents of **R1** onto the stack.
5. Push the contents of **R2** onto the stack.
6. Push the contents of **R3** onto the stack.
7. Push the contents of **R4** onto the stack.
8. Single-step through each instruction.
9. Observe the **Stack Pointer (SP)** and stack memory after each PUSH instruction.
10. Stop the program.

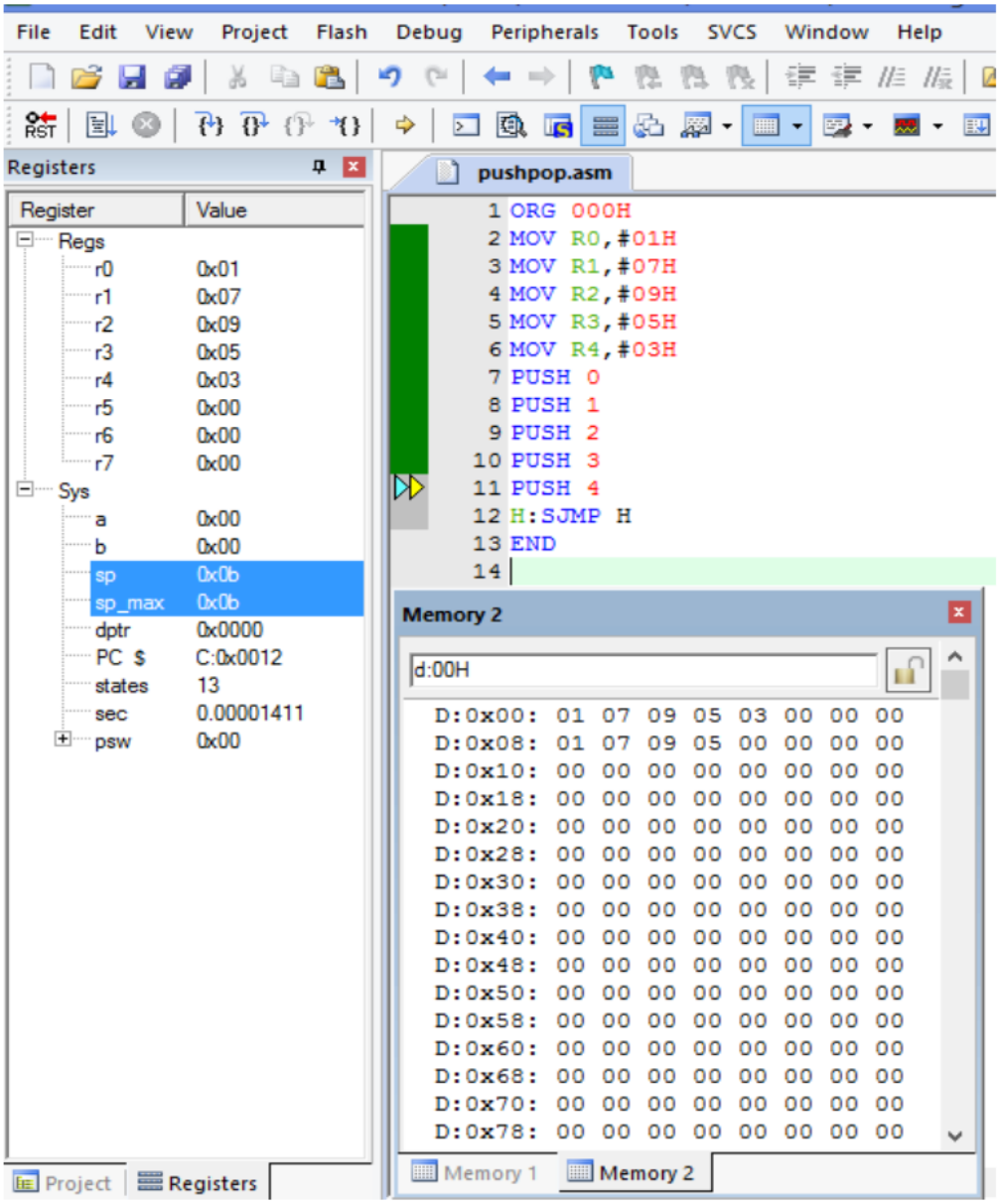
4. Flow Chart



CODE(HAND WRITTEN):-


Date ___ / ___ / _____
Program 2 :-
2) ORG 000H
MOV R0, # 01H
MOV R1, # 07H
MOV R2, # 09H
MOV R3, # 05H
MOV R4, # 03H
push 0
push 1
push 2
push 3
push 4
END

SIMULATE:-



Observation / Inference / Result

Observation:

- Registers **R0 to R4** are loaded with respective data values.
- On execution of each **PUSH instruction**, the **SP register increments by 1**.
- The pushed data appears in the stack memory starting from **08H** (default SP location).

Inference:

- Stack operation in 8051 follows **Last In First Out (LIFO)** principle.
- Each PUSH instruction stores data at the address pointed by **SP** after increment.

Result:

Thus, the **8051 ALP to load registers R0–R4 and push their contents onto the stack was successfully executed and verified using Keil software**.

Question 3

1. Aim (Statement)

To write and execute an **8051 Assembly Language Program (ALP)** to **initialize the Stack Pointer (SP) to 0DH**, store different values in RAM locations **08H to 0DH**, **pop the stack contents into registers R0 to R4**, and **single-step the program to observe the stack and SP register using Keil software**.

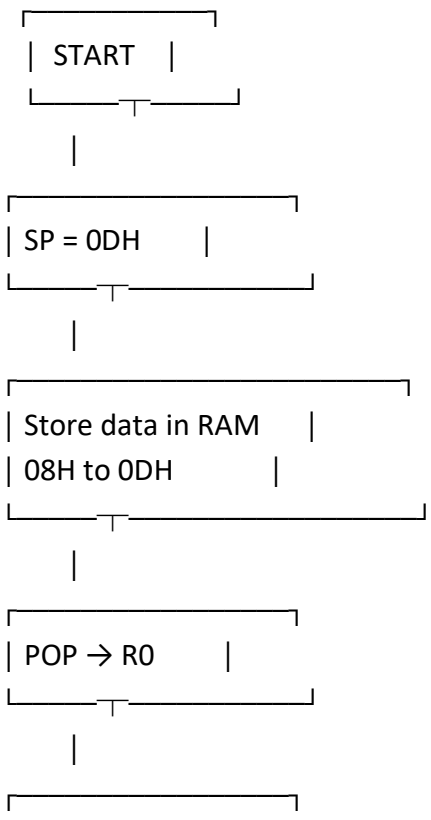
2. Apparatus Required

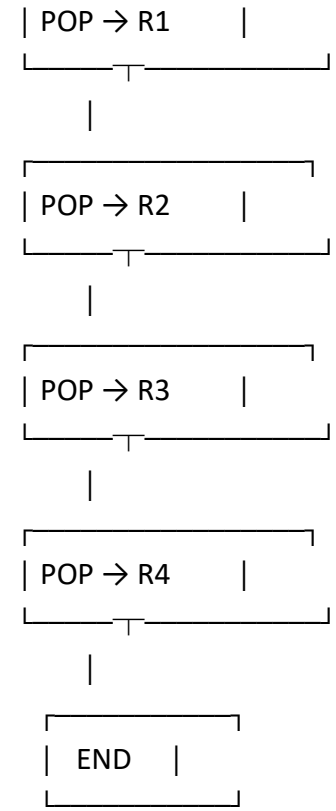
1. Personal Computer (PC)
2. Keil µVision Software
3. 8051 Microcontroller Simulator (Keil built-in simulator)
4. 8051 Instruction Set
5. Power supply (if hardware kit is used)

3. Algorithm

1. Start the program.
2. Initialize the **Stack Pointer (SP) to 0DH**.
3. Store different data values in RAM locations **08H, 09H, 0AH, 0BH, 0CH, and 0DH**.
4. Pop the contents of the stack into registers in the following order:
    a. POP into R0
    b. POP into R1
    c. POP into R2
    d. POP into R3
    e. POP into R4
5. Single-step through each POP instruction.
6. Observe the **Stack Pointer (SP)** and stack memory after each POP.
7. Stop the program.

4. Flow Chart



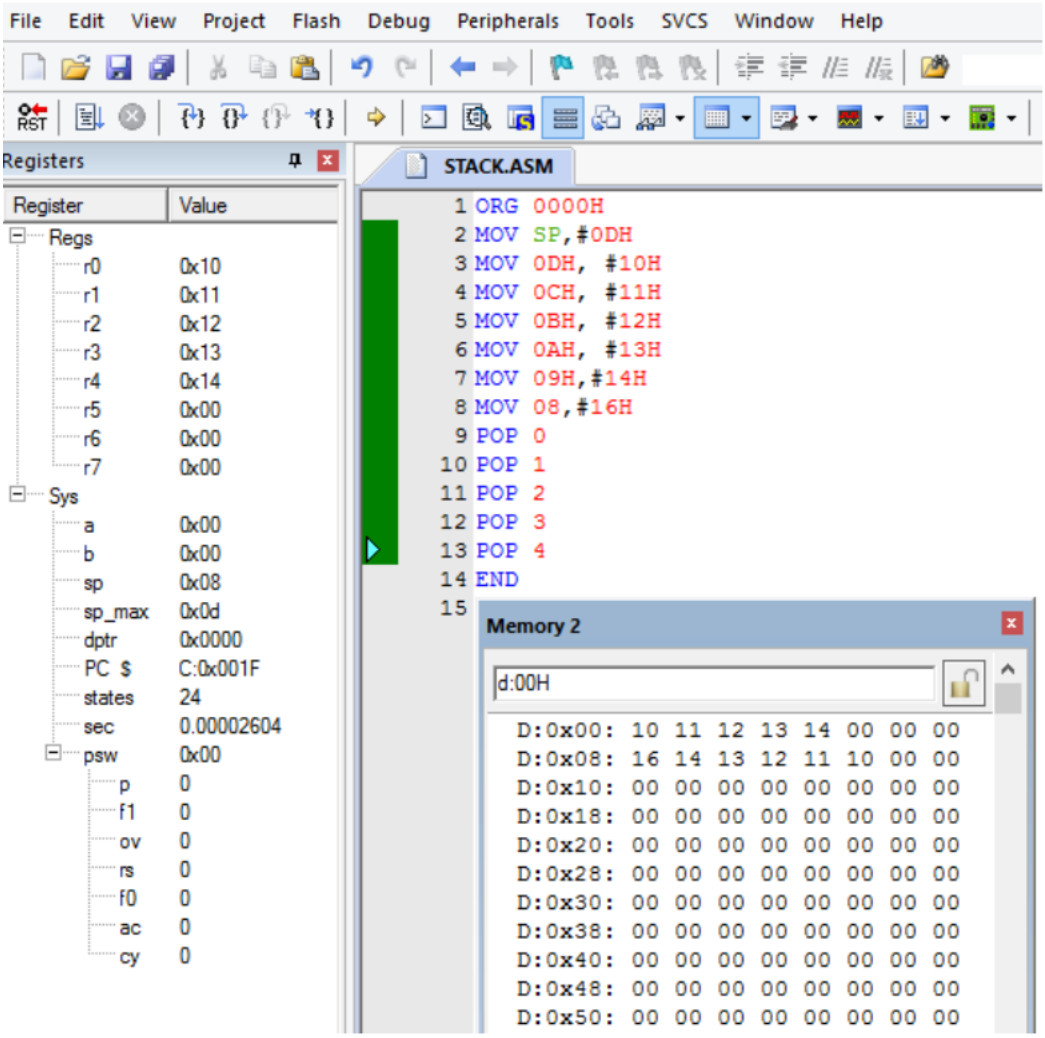


CODE(HANDWRITTEN):-

program - 3 :-

```
ORG 0000H
MOV SP, #0DH
MOV 0DH, #10H
MOV 0CH, #11H
MOV 0BH, #12H
MOV 0AH, #13H
MOV 09H, #14H
MOV 0H, #16H
 pop 0
 pop 1
 pop 2
 pop 3
 pop 4
END
```

SIMULATION:-



Observation:

- The Stack Pointer is initialized to **0DH**.
- Data values stored in RAM locations **08H to 0DH** are popped in **reverse order**.
- After each POP instruction, the **SP register decrements by 1**.
- Registers **R0 to R4** contain the popped stack values.

Inference:

- The 8051 stack follows the **Last In First Out (LIFO)** principle.
- POP instruction retrieves data from the stack and updates the Stack Pointer automatically.

Result:

Thus, the **8051 ALP to initialize SP, store data in stack memory, and pop stack contents into registers R0–R4 was successfully executed and verified using Keil software**.

Question 4

1. Aim (Statement)

To write, assemble, and simulate **8051 Assembly Language Programs (ALP)** to perform **basic data transfer operations** between different memory spaces using **Keil µVision software**, namely:

1. Internal RAM to Internal RAM
2. Internal RAM to External Memory
3. External Memory to Internal RAM
4. External Memory to External Memory
5. Internal ROM to Internal RAM

2. Apparatus Required

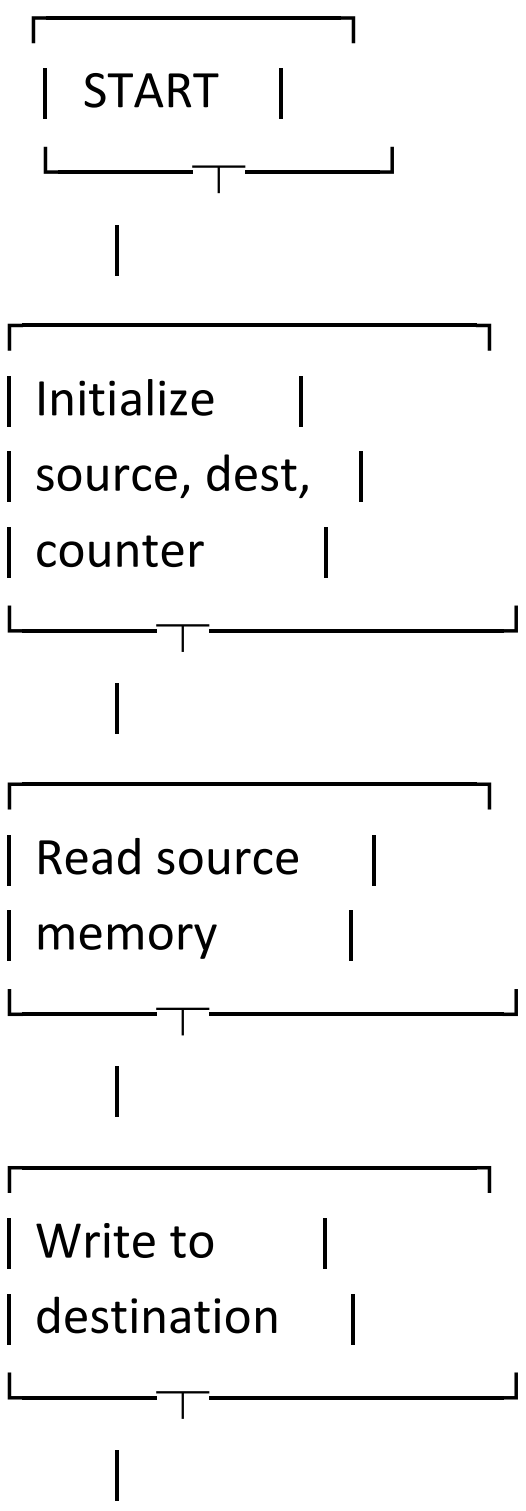
1. Personal Computer (PC)
2. Keil µVision Software
3. 8051 Microcontroller Simulator (Keil built-in simulator)
4. 8051 Instruction Set
5. External RAM model (Keil simulation)

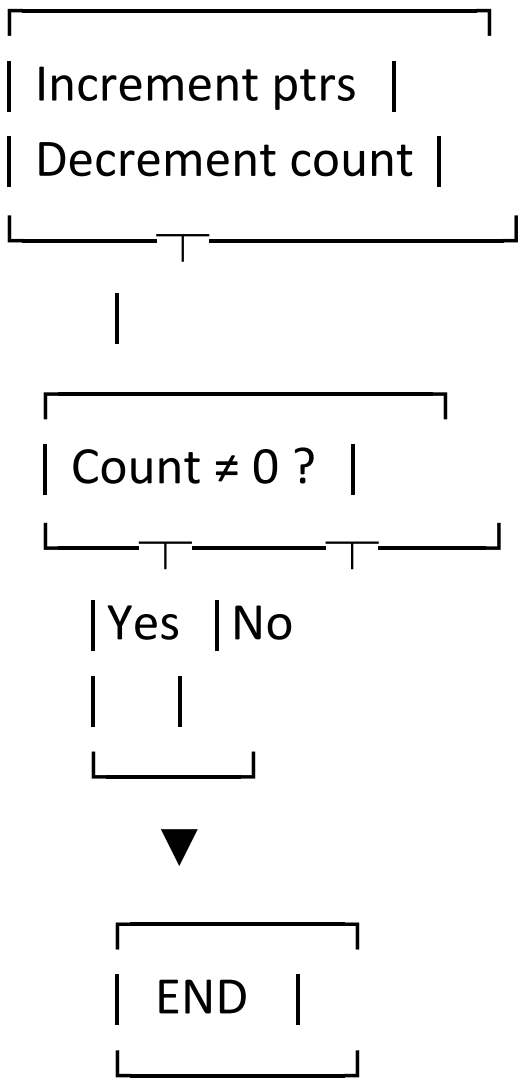
3. Algorithm

General Algorithm (Applicable to all cases)

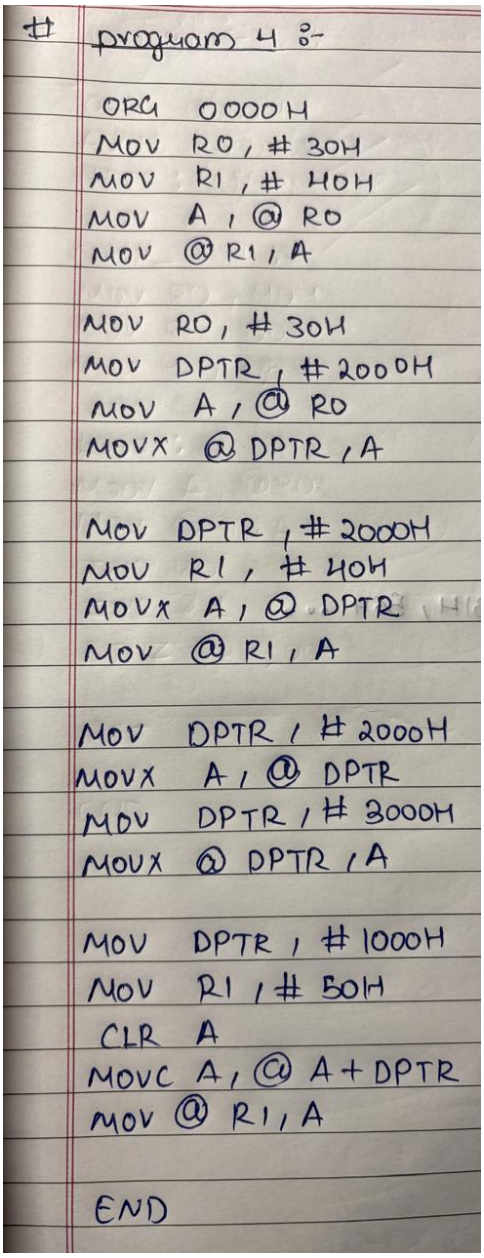
1. Start the program.
2. Initialize source and destination addresses based on the type of transfer.
3. Initialize a counter with number of bytes to be transferred.
4. Read data from source memory.
5. Write data to destination memory.
6. Increment source and destination pointers.
7. Decrement counter.
8. Repeat until counter becomes zero.
9. Stop the program.

4. Flow Chart

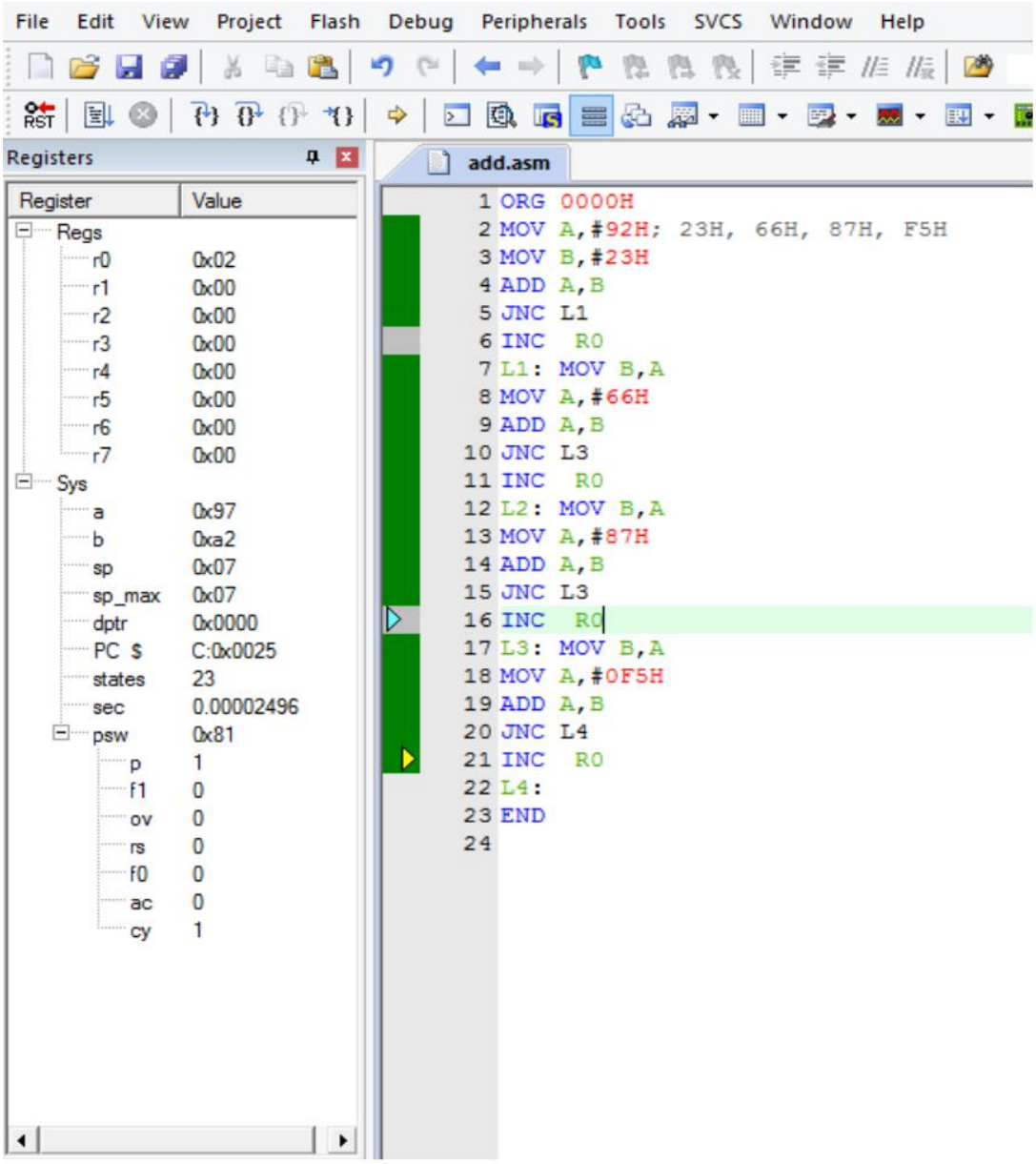

START
Initialize source, dest, counter
Read source memory
Write to destination



CODE(HANDWRITTEN):-


program 4 :-
ORG 0000H
MOV R0, #30H
MOV R1, #40H
MOV A, @R0
MOV @R1, A
MOV R0, #30H
MOV DPTR, #2000H
MOV A, @R0
MOVX @DPTR, A
MOV DPTR, #2000H
MOV R1, #40H
MOVX A, @DPTR
MOV @R1, A
MOV DPTR, #2000H
MOVX A, @DPTR
MOV DPTR, #3000H
MOVX @DPTR, A
MOV DPTR, #1000H
MOV R1, #50H
CLR A
MOVC A, @A+DPTR
MOV @R1, A
END

SIMULATE:-



Observation:

- Data is correctly transferred between internal RAM, external memory, and internal ROM.
- MOV, MOVX, and MOVC instructions perform expected memory transfers.
- Source and destination pointers increment correctly.

Inference:

- MOV is used for internal RAM transfers.
- MOVX is used for external memory access.
- MOVC is used for code (ROM) memory access.
- The 8051 supports flexible data movement across memory spaces.

Result:

Thus, the **basic data transfer operations between internal RAM, external memory, and internal ROM on the 8051 microcontroller were successfully implemented and verified using Keil µVision software**.

QUESTION –5:-

1. Aim (Statement)

To write, assemble, and simulate an **8051 Assembly Language Program (ALP)** to **add 10 bytes of data stored in ROM starting at address 200H**, by first transferring the data from **ROM to internal RAM**, performing the addition, and **storing the final 16-bit result in registers R2 (LSB) and R3 (MSB)** using **Keil µVision software**, and to single-step the program to examine the data.

2. Apparatus Required

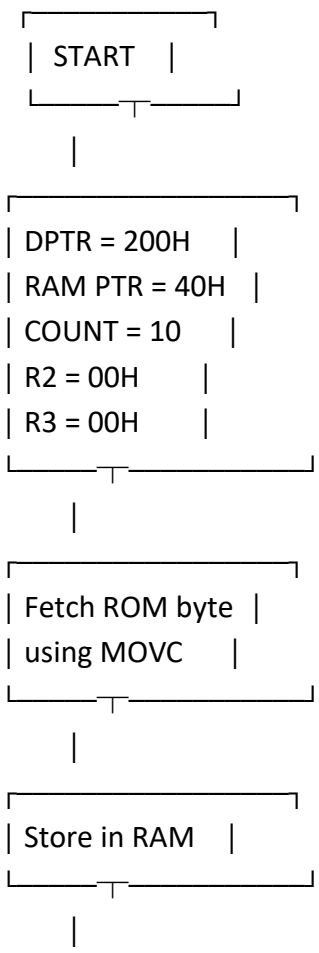
1. Personal Computer (PC)
2. Keil µVision Software
3. 8051 Microcontroller Simulator (Keil built-in simulator)
4. 8051 Instruction Set

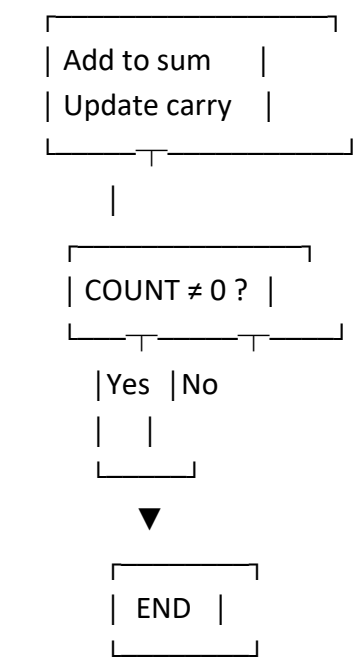
3. Algorithm

1. Start the program.

2. Initialize DPTR to point to ROM address **200H**.
3. Initialize a RAM pointer to store data temporarily in internal RAM.
4. Initialize a counter for **10 bytes**.
5. Clear accumulator and registers **R2 and R3** for sum storage.
6. Fetch each byte from ROM using MOVC.
7. Store the fetched byte into internal RAM.
8. Add the RAM data to the accumulator.
9. If carry occurs, increment **R3** (MSB).
10. Store the accumulator result in **R2** (LSB).
11. Repeat until all 10 bytes are added.
12. Stop the program.

4. Flow Chart





CODE (HANDWRITTEN):-

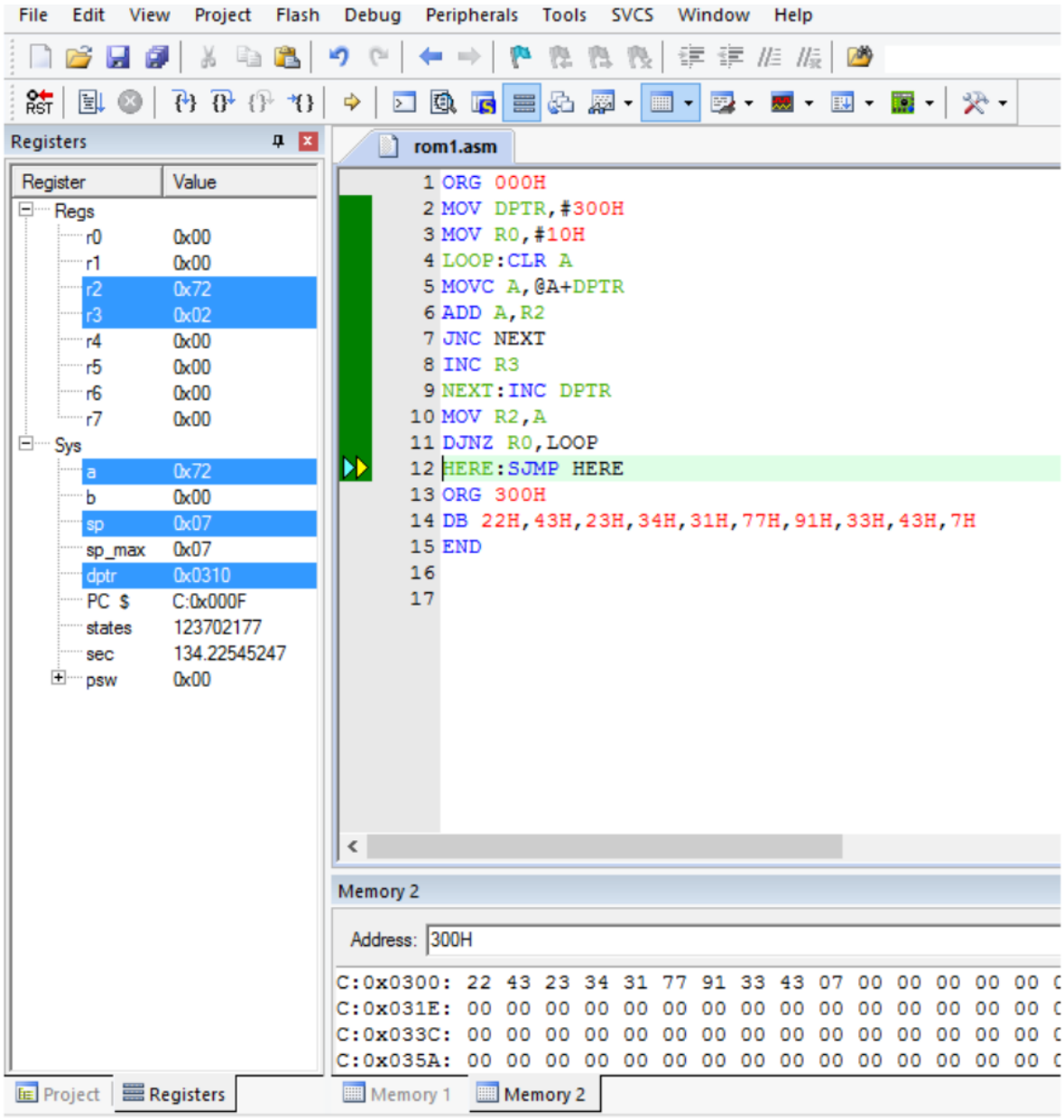
Date __/__/____

program - 5 :-

2)

```
ORG 000H
MOV DPTR, #300H
MOV R0, #10H
LOOP : CLR A
MOVC A, @ A+DPTR
ADD A, R2
JNC NEXT
INC R3
NEXT : INC DPTR
MOV R2, A
DJNZ R0, LOOP
HERE : SJMP HERE
ORG 300H
DB 92H, 34H, 84H, 81H, 56H
END
```

SIMULATION:-



Observation:

- Data bytes stored in ROM starting at **200H** are correctly fetched using MOVC.
- The fetched data is stored in internal RAM before addition.
- The sum of 10 bytes is stored correctly as a **16-bit value**, with **R2 holding the LSB** and **R3 holding the MSB**.

Inference:

- ROM data cannot be added directly and must be transferred to RAM.
- Carry handling is necessary when adding multiple bytes.
- The 8051 supports multi-byte arithmetic using registers and carry flag.

Result:

Thus, the **8051 ALP to add 10 bytes of data stored in ROM and store the result in registers R2 and R3 was successfully executed and verified using Keil µVision software**.

QUESTION-6:-

1. Aim (Statement)

To write, assemble, and simulate an **8051 Assembly Language Program (ALP)** to implement the given **digital logic circuit**, generate the logic function **F**, and verify the output by **single-stepping the program using Keil µVision software**.

2. Apparatus Required

1. Personal Computer (PC)
2. Keil µVision Software
3. 8051 Microcontroller Simulator (Keil built-in simulator)
4. 8051 Instruction Set

3. Logic Description of the Given Circuit

From the given diagram:

- Inputs: **A, B, C, D**
- Gates used:
    - AND gate
    - OR gate

    - NOT gate
    - NAND gate
- Output: **F**

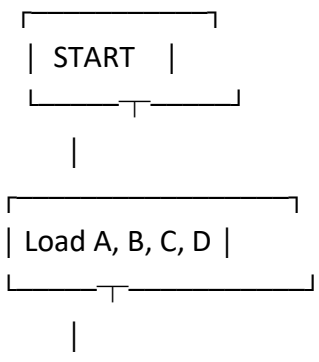
Logic Expression

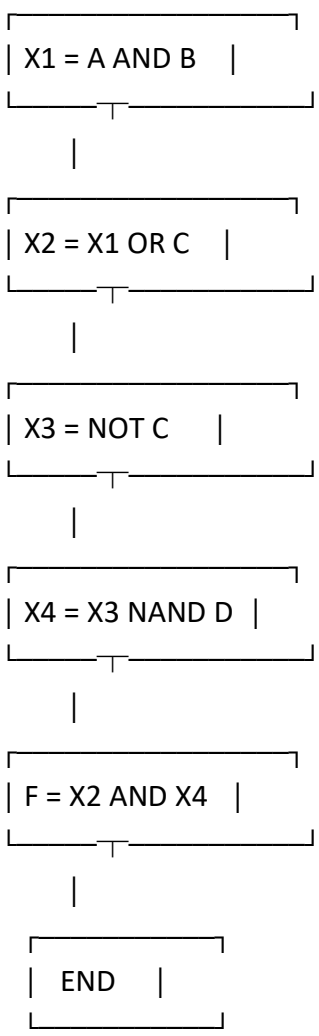
1. X1 = A AND B
2. X2 = X1 OR C
3. X3 = NOT C
4. X4 = X3 NAND D
5. F = X2 AND X4

4. Algorithm

1. Start the program.
2. Load input values **A, B, C, D** into registers.
3. Perform **AND operation** between A and B.
4. Perform **OR operation** between the result and C.
5. Perform **NOT operation** on C.
6. Perform **NAND operation** between NOT C and D.
7. Perform **AND operation** between the two intermediate results.
8. Store the final output in a register.
9. Stop the program.

5. Flow Chart





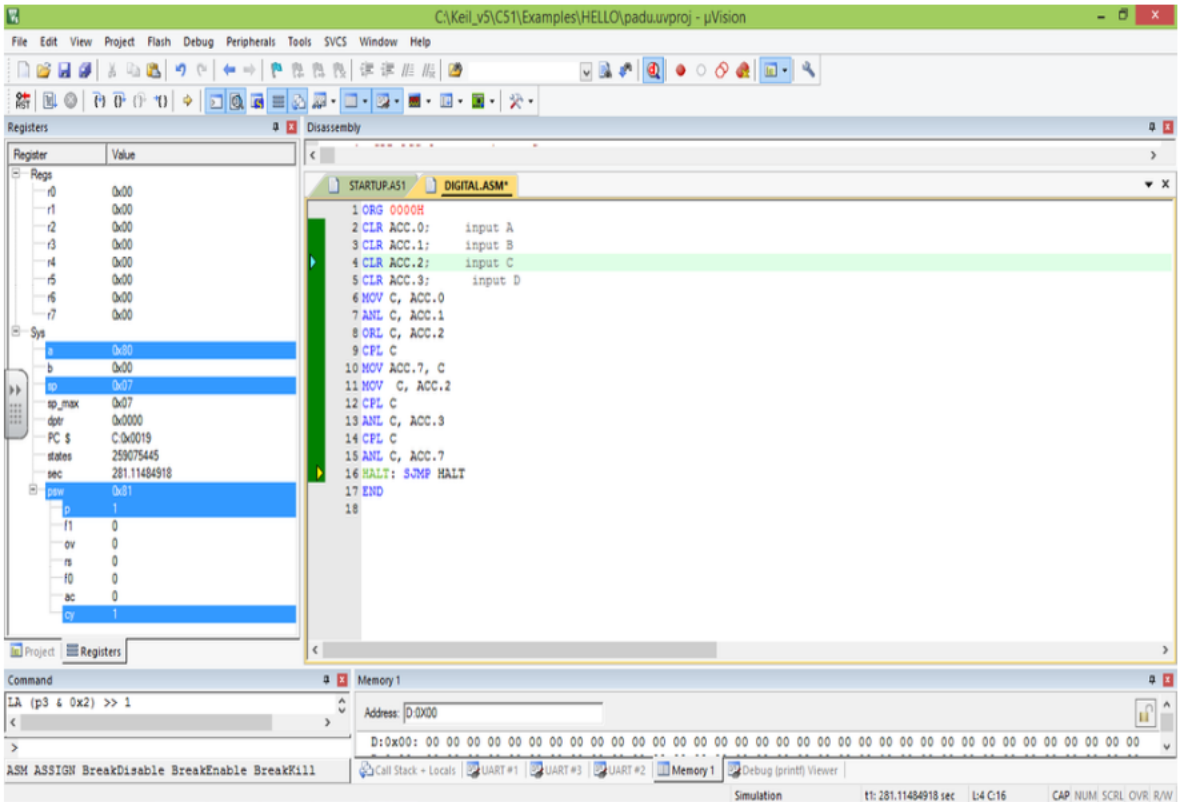
CODE(HANDWRITTEN):-

program - 6 :-

2)

```
ORG 0000H
SETB ACC.0 ;
SETB ACC.1 ;
CLR ACC.2 ;
SETB ACC.3 ;
MOV C, ACC.0
ANL C, ACC.1
ORL C, ACC.2
CPL C
MOV ACC.7, C
MOV C, ACC.2
CPL C
ANL C, ACC.3
CPL C
ANL C, ACC.7
HALT : SJMP HALT
END
```

SIMULATION:-



Observation:

- Input values are correctly loaded into registers.
- Logical operations match the behavior of the given digital logic circuit.
- Final output **F** is generated correctly and stored in register **R6**.

Inference:

- Digital logic circuits can be implemented using basic 8051 logical instructions.
- The 8051 supports AND, OR, NOT, and NAND logic using bitwise operations.

Result:

Thus, the **8051 Assembly Language Program for the given digital logic circuit was successfully executed and verified using Keil µVision software**.